

Prise en main d'un environnement de C embarqué

Configurez une cible, compilez votre programme et exécutez-le

Dans ce chapitre, vous allez apprendre les bases pour utiliser μ Vision. Vous commencerez par configurer un nouveau projet puis vous écrirez quelques lignes d'un programme avant de le compiler et de l'exécuter. Vous verrez comment réaliser cette exécution en simulation, c'est-à-dire en exécutant le programme sur votre machine en simulant l'architecture du microcontrôleur STM32, mais aussi sur la cible réelle, c'est-à-dire directement sur la carte NUCLEO.

Configurer votre projet

Démarrez le programme Keil μ Vision.

Pour se lancer dans l'écriture de code sous μ Vision, la première chose à faire est de créer un projet et de le configurer.

Pour créer un projet, il vous suffit de sélectionner *New μ Vision Project* dans le menu *Project*, puis de suivre les différentes étapes en faisant attention aux éléments suivants :

- choisir la bonne puce : lors de la création du projet, ou en sélectionnant *Options for target* (🔧) puis l'onglet *Device*. Ensuite, vous devez choisir le microcontrôleur *STM32F103RB* dans la liste *STMicroelectronics* puis dans la série *STM32F1 serie*.
- inclure les fichiers de démarrage : les fichiers de démarrage sont sélectionnés lors de la création du projet ou a posteriori via le *Manage Run-Time Environment* (🔧). Il faut au minimum cocher *startup* dans *Device* et *CORE* dans *CMSIS*.

Vous devriez avoir dans le répertoire *Device* du projet les fichiers *startup_stm32f10x_md.s* et *system_stm32f10x.c*.

Faites une petite pause (danser, chanter, etc.) avant de passer à la suite...

Écrire et compiler votre premier code

Une fois le projet construit, il s'agit maintenant d'y ajouter vos premières lignes de code puis de les compiler. Vous allez simplement écrire une procédure *main* et réalisez le « Hello World » du programmeur de carte en faisant clignoter une LED. Pour cela, copiez simplement le code ci-dessous (vous verrez les détails dans la suite du cours).

```
#include "STM32f10x.h"
```

```
int main (void)
```

```
{
```

```

RCC->APB2ENR |= RCC_APB2ENR_IOPAEN ;

GPIOA->CRL = GPIOA->CRL & ~(0xF << 4*5);

GPIOA->CRL = GPIOA->CRL | (0x01 << 4*5);

while(1)

{

    GPIOA->ODR ^= (1 << 5);

}

return 0;

}

```

Pour ajouter un nouveau fichier, sélectionnez *New* dans le menu *File* ou bouton *New* () puis enregistrez le fichier avec la bonne extension (.c ou .h). Ensuite, ajoutez le fichier au projet (sinon il ne sera pas compilé) : allez dans l'arborescence du projet, faites un clic droit sur le dossier des sources et choisissez *Add New Files to Group* (ou double clic sur le répertoire contenant les sources du projet). Vous pouvez aussi faire cela en passant par *Manage Project Items* ()

La compilation et l'édition des liens est réalisée en cliquant sur les boutons *Build* () ou *Rebuild* () ou en allant dans le menu *Project* et en choisissant *Build target* ou *Rebuilt all target files*. Le bouton *Translate* () procède uniquement à la compilation du fichier courant. Maintenant que cela est fait, respirez un grand coup et passez à la suite.

Simuler le programme

Vous avez réussi à compiler votre premier programme, maintenant vous allez l'exécuter. Avant de le faire sur une cible réelle, vous le ferez en mode simulé. Cela est très pratique pour déboguer et mettre au point vos applications car vous pourrez contrôler l'avancement du système.

Vous verrez ainsi un outil indispensable pour le débogage : l'exécution du programme pas à pas afin de bien observer le comportement du système et ainsi trouver une éventuelle source de bug. Avec µVision vous allez voir que plusieurs moyens s'offrent à vous pour réaliser cela.

Remarquez que vous pourrez faire exactement les mêmes manipulations sur la cible réelle et donc "geler" le système pour en observer son état. La grande différence avec l'exécution en

mode simulé est que le temps physique ne peut pas être arrêté. Certains périphériques pourront aussi continuer à évoluer alors que la CPU (Central Processing Unit, le processeur) a été figée. Il devient complexe d'observer certains effets liés au déroulement continu du temps (synchronisation par exemple).

Pour exécuter le programme en simulation, allez dans configuration du projet *Options for target* () , onglet *Debug* et assurez-vous que l'option *Use Simulator* est bien cochée.

Il suffit ensuite de lancer la session de debug ().

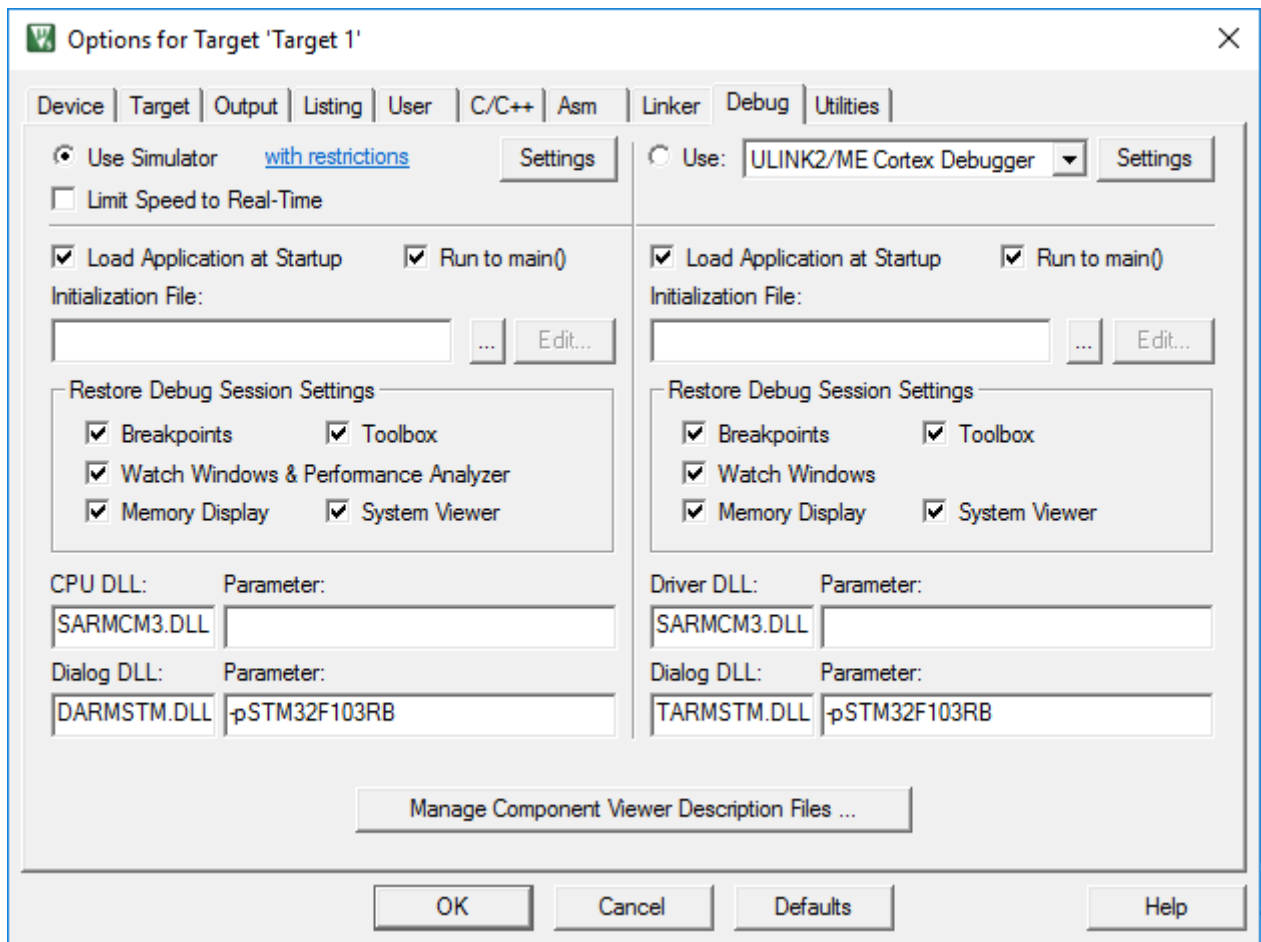
Si vous avez une erreur lors du lancement du simulateur, ou si lors de la simulation vous avez des message d'erreur de type *Access violation*, allez dans la configuration du projet () , onglet *Debug*. Vérifiez que dans la partie à gauche qui sert pour la simulation, vous avez bien les valeurs suivantes pour :

- CPU DLL : SARMCM3.DLL
- Dialog DLL : DARMSTM.DLL Parameter : -pSTM32F103RB

Si vous avez bien tout installé, et en particulier le *MDK v4 Legacy Support for ARM Cortex-M devices*, vous devriez avoir ces options automatiquement. **Sinon écrivez ce valeurs dans les champs** (voir capture d'écran ci-dessous).

La partie droite sert pour l'exécution sur cible réelle et vous aurez besoin des valeurs suivantes :

- Driver DLL : SARMCM3.DLL
- Dialog DLL : TARMSTM.DLL Parameter : -pSTM32F103RB



L'onglet Debug

Une fois le debug lancé, vous pouvez exécuter le code avec le bouton *Run* (🏃) et l'arrêter avec le bouton *Stop* (⏹). Si vous voulez ré-initialiser le système comme lors d'un redémarrage, vous pouvez utiliser le bouton *Reset* (🔄).

Le contrôle de l'exécution en pas à pas se fait à l'aide des boutons *Step* (🔍), *Step Over* (🔍➡) et *Step Out* (🔍➡➡). Le bouton *Step* permet d'avancer d'une instruction, le bouton *Step Over* évite d'entrer dans une fonction et *Step Out* va à l'instruction suivant la sortie d'une fonction. Le bouton *Run to the cursor line* (🏃➡) permet, comme son nom l'indique, de provoquer l'exécution jusqu'à atteindre la ligne de code sélectionné.

L'ajout d'un point d'arrêt se fait soit en double cliquant à droite de la ligne où l'on souhaite déposer le point d'arrêt, soit en cliquant sur le bouton *Insert/Remove Breakpoint* (🔴). En utilisant le bouton *Run* (🏃), l'exécution avance jusqu'au prochain point d'arrêt.

Vous avez réussi à exécuter votre premier programme, c'est parfait. Vous pouvez prendre un peu de repos et aller faire un peu d'exercice avant de vous lancer dans l'exécution sur une cible réelle.

Exécuter le programme sur une cible

Si vous avez fait l'acquisition d'une carte Nucleo-F103RB, vous pouvez exécuter votre code dessus. Pour cela, il vous faut définir les paramètres de la cible avant de charger le code dessus.

Pour configurer l'exécution sur une cible réelle, vous pouvez créer une nouvelle cible via le *Manage Project Items* (📁).

Modifiez ensuite les options de configuration (⚙️).

Dans l'onglet *Debug*, cochez *Use* dans la partie droite choisissez le driver ST-Link Debugger puis cliquez sur *Settings*. Vous pourrez vérifier que la carte est bien reconnue (n'oubliez pas de brancher la carte) et que le protocole utilisé est le SW (champ Port).

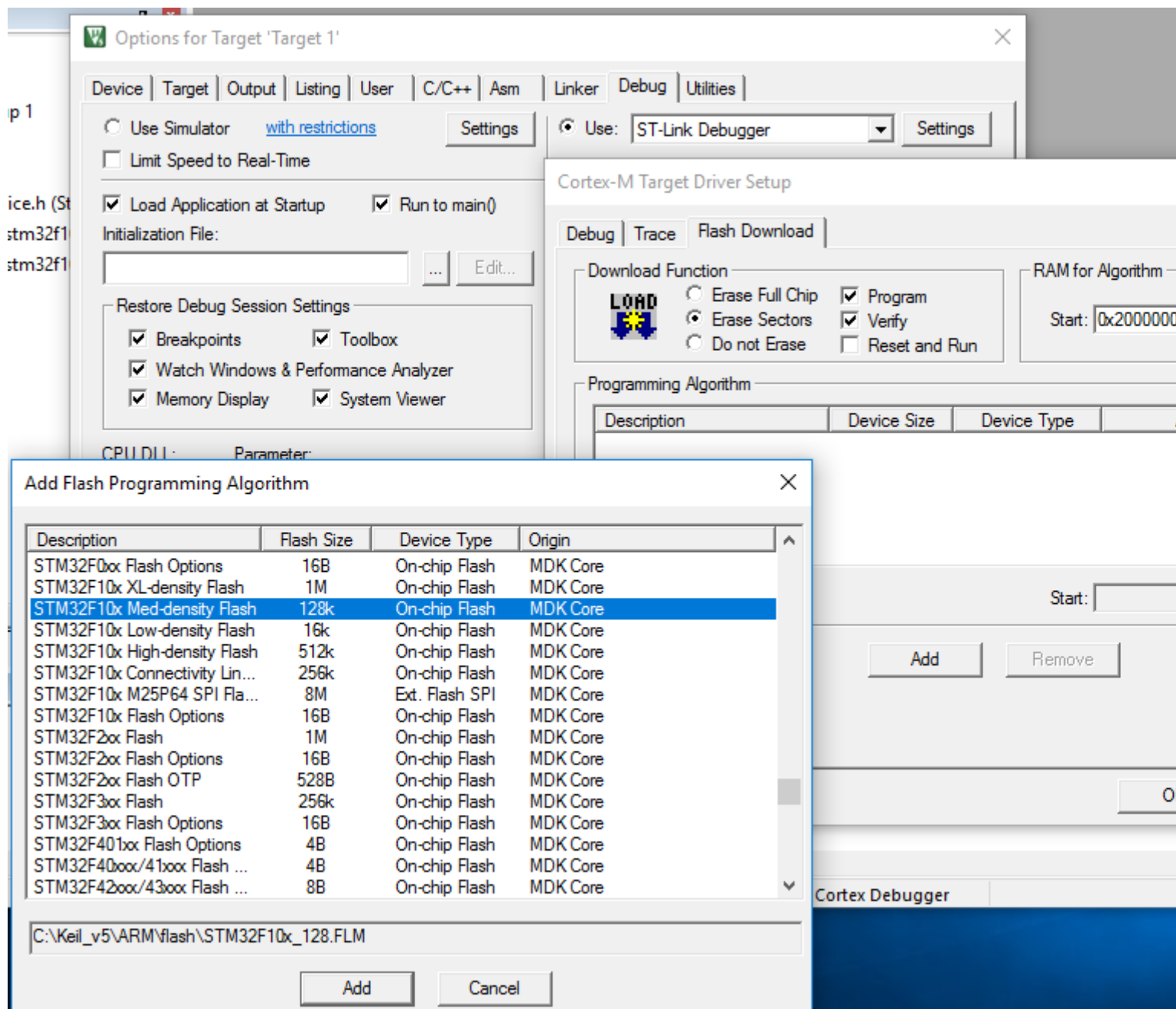
Si des erreurs persistent lors du chargement du code sur la carte, tentez de basculer le protocole en JTAG puis de le remettre en SW.

Vérifiez que les DLL sont définies correctement (voir la section précédente).

- Pour charger le code sur le carte, il suffit de cliquer sur *Download* (📶).
- Pour exécuter le code, il faut lancer la session de debug (🔍) et faire la même chose qu'en simulé.

Remarquez que si vous lancez directement le debug, le chargement du code se fait automatiquement sans avoir besoin de passer par l'étape *Download* (📶).

Si vous avez un message de type "Error: Flash Download failed", allez dans les options du projet (⚙️) puis dans l'onglet *Debug*. Dans la partie droite qui sert pour l'exécution sur cible réelle, cliquez sur *Settings*, onglet *Flash Download* et cliquez sur *Add* et choisissez l'algorithme **STM32F10x Med-density Flash**. Si tout s'est bien passé vous devriez voir l'algorithme dans la liste des *Programming Algorithm* de la fenêtre *Cortex-M Target Driver Setup*.



Installer l'algorithme STM32F10x Med-density Flash

Si la LED nommée LD2 sous les boutons poussoirs s'est allumée, c'est que vous êtes prêt pour la suite ! Si vous ne l'avez pas fait clignoter pas à pas, c'est que vous êtes allé trop vite...

Une fois le code chargé sur la carte, il reste en mémoire et s'exécutera dès que la carte est alimentée. Vous pouvez le constater en fermant µVision et en rebranchant la carte via l'USB (le câble servant alors uniquement d'alimentation). Vous verrez la LED LD2 s'allumer ce qui signifie que le code a bien été exécuté sans avoir besoin de µVision. Par contre, vous ne pouvez plus contrôler l'avancement pas à pas du code: son exécution est complètement autonome.